Adjacency: Compiling Brand-Safety Intent into Typed Evidence and Fail-Closed Decisions

Arun Sharma [1]

Abstract

X gives advertisers direct adjacency controls over where promoted content appears, but advertiser intent begins as prose while serving controls ultimately require discrete, reviewable decisions. ADJACENCY compiles that prose into a typed and hashed policy, evaluates multimodal inventory with a cost-gated reasoning ladder, and admits a verdict only after deterministic checks validate its source clauses, evidence, action, and confidence. The same prose also generates the keyword blocklist used for comparison, which avoids giving the proposed engine a richer intent signal than the baseline. A frozen-corpus study establishes three concrete properties rather than a broad accuracy claim: seeded structural faults are rejected, an unconstrained raw-prompt baseline is poorly grounded and not fully stable across identical inputs, and selective escalation has a measurable cost. A live durable human-review transition completes the failure path. The system is a read-only autopsy and shadow-evaluation tool. It is not an ad-serving integration, an estimate of X's internal performance, or a synthetic-media detector.

1. Introduction

Advertising was Twitter's largest disclosed revenue surface before the company became private (Twitter, Inc., 2022). X's current documentation still places advertiser-controlled adjacency at the center of brand safety. It lets advertisers exclude chosen keywords from neighboring placements (X Corp., 2026). That control is practical, legible, and fast. It also turns a sentence-level policy into a lexical filter. A safe discussion can contain an excluded word. A risky image can accompany clean text. Both errors matter because one withholds monetizable inventory and the other exposes an advertiser to content it intended to avoid.

The tempting replacement is a multimodal prompt. It can read the prose and inspect the image, then return ALLOW, REVIEW, or BLOCK. That gains context but loses a stable execution contract. The model can cite text at the wrong offset, refer to a policy clause that does not exist, disagree with its own severity field, or change behavior under the same input. A polished rationale does not repair any of those failures. In a serving-adjacent system, prose should never be load-bearing.

ADJACENCY starts from a design discipline used in Spatial Atlas: represent perception as typed scene state, place deterministic checks after model output, and spend more reasoning only when confidence or consistency says it is needed. Here the scene is not geometric. It binds an advertiser's `PolicySpec`, one `InventoryItem`, structured evidence, and a `Verdict`. The architecture points the same discipline at X's historically largest disclosed revenue surface.

The central design choice is to separate generation from admission. A model may propose a policy or a verdict. It cannot decide whether its own proposal is valid. Pure functions do that after the call. A gate failure moves the decision toward withholding, usually REVIEW, and writes a stable code to the audit ledger. The rationale remains visible to a person but invisible to every gate.

Contributions. This report presents five connected pieces.

1. A policy compiler maps advertiser prose to an immutable, hashed `PolicySpec`. Each clause points back to the exact characters that authorized it. Compile-time G0 rejects an invented or misplaced source span. The implementation is in `src/adjacency/policy.py` and `src/adjacency/gates.py`.
2. A typed multimodal verdict carries clause identifiers, exact text spans or image boxes, severity, action, confidence, and reasoning tier. Deterministic gates validate those fields and never read the generated rationale. The contract is in `src/adjacency/contracts.py`.
3. A selective ladder routes clean text through deterministic Tier 0, uses low reasoning at Tier 1, and esca-

[1]Department of Computer Science, University of Minnesota, Twin Cities, Minneapolis, MN, USA. Correspondence to: Arun Sharma <sharm636@umn.edu>.

Technical report prepared for the xAI Grokathon, August 2026. Not an ICML publication.

lates uncertain or structurally risky items to high reasoning at Tier 2. Its measured routing and cost are recorded in `artifacts/wednesday/judge_report.json`.

4. A fair comparison baseline is generated from the exact same advertiser prose, then matched deterministically. The Autopsy separates agreement from over-blocks and under-blocks, with REVIEW treated as withholding. The source artifacts are `artifacts/tuesday/baseline_blocklist.json` and `artifacts/ui/autopsy_snapshot.json`.

5. A seeded fault study, a repeated raw-prompt study, and a real Temporal Cloud transition test the failure path directly. Their records are listed in Table 1.

The claim is deliberately narrow. This is evidence that typed admission gates catch the injected failures they target, that the tested prompt baseline has a grounding problem, and that the tested ladder has a measurable cost profile. It is not evidence of production accuracy. No gold-label accuracy figure is reported.

2. Context and Related Work

Advertiser controls. X documents both platform protections and advertiser controls. Its Adjacency Controls use keyword and account exclusions and explicitly note that placement protection is not guaranteed to be perfectly effective (X Corp., 2026). ADJACENCY does not claim to reproduce X's internal blocklist, ranking stack, or policy models. The baseline is Arun's construction. It is generated from the same prose as the compiled policy, then visibly labeled as such in `artifacts/tuesday/baseline_blocklist.json`. This keeps the comparison about representation: identical intent expressed as keywords versus as a compiled policy.

Multimodal safety. The Hateful Memes Challenge demonstrates why independent text and image signals can be insufficient. Its counterfactual construction changes the joint meaning while preserving a benign-looking modality (Kiela et al., 2020). Brand suitability is not identical to hate-speech detection, but the representational warning transfers directly. Text-only keyword matching cannot inspect an image region. ADJACENCY therefore accepts text evidence and image bounding boxes under one verdict schema.

Selective prediction and human review. Selective classification gives a predictor an explicit reject option and studies the resulting risk-coverage tradeoff (Geifman & El-Yaniv, 2019). Learning-to-defer work makes the downstream expert part of the decision system rather than an exception path (Mozannar & Sontag, 2020). Content-moderation work extends that framing to delayed and capacity-limited review queues (Lykouris & Weng, 2024). ADJACENCY uses the same operational principle without claiming a learned deferral policy. Confidence, media, boundary severity, low-tier gate failure, model failure, and near-duplicate disagreement are explicit escalation predicates. A REVIEW result enters a human queue.

Cascades and behavioral evaluation. Model cascades trade cost against capability by routing easy requests away from expensive calls (Chen et al., 2023b). Here the first stage is not a smaller language model. It is a deterministic decider. The remaining stages use the same model at different reasoning effort, which xAI exposes directly for Grok 4.5 (xAI, 2026a;b). Behavioral testing argues for targeted capability and invariance checks beyond one aggregate score (Ribeiro et al., 2020). The synthetic fault suite follows that logic. Each mutation has an expected gate code, so an unrelated rejection does not count as success.

Model drift and replay. Hosted model behavior can change while the API name stays fixed (Chen et al., 2023a). Every external request in ADJACENCY is therefore content-addressed. Recording is explicit. Offline replay is the default. Corpus, policy, request, and response hashes make the reported run inspectable without requiring a live model endpoint.

3. System Design

3.1. Five-part skeleton

The architecture follows the five-part teaching skeleton in Spatial Atlas: problem contract, typed interaction map, module responsibilities, end-to-end flow, and explicit design decisions. The runtime form is shown in Figure 1.

3.2. Policy as a compiled artifact

Let advertiser prose be a normalized string p. The compiler proposes a policy $S = (v, a, p, C)$ with version v, advertiser a, and clauses C. Each clause c stores a source interval $[s_c, e_c)$ and quoted text q_c. Admission requires

$$p[s_c : e_c] = q_c \quad \text{for every } c \in C. \tag{1}$$

The comparison is exact after Unicode NFC normalization. Offsets are Python character indices, not byte offsets and not UTF-16 code units. This choice matters for emoji and combining marks. G0 runs immediately after the high-reasoning structured call. The trusted advertiser name, normalized prose, and requested policy version must also equal the returned values. Only then is the policy cached. The compiled instance and policy hash are stored in `artifacts/tuesday/policy_spec.json`.

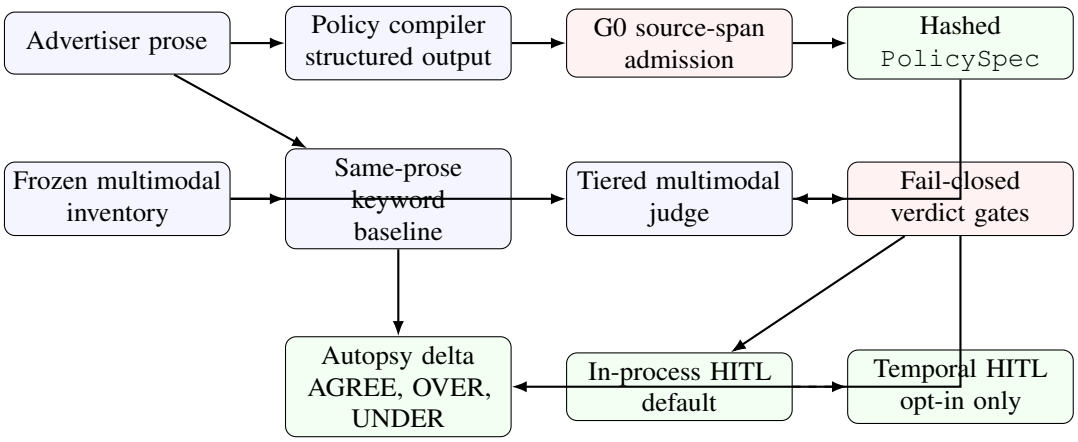

Figure 1. ADJACENCY separates proposal from admission. Model-produced structures pass through deterministic checks before they can become decisions. The default demo ends at the in-process queue. Temporal is an opt-in durability adapter and is not imported by the demo path. Source: `src/adjacency/ui.py`, `src/adjacency/hitl.py`, and `src/adjacency/temporal_hitl.py`.

The same p enters a separate keyword-expansion call. Trusted code, not the model, attaches the advertiser, version, policy hash, and original prose to the returned terms. Terms are normalized, deduplicated, sorted, and capped at the product limit documented by X. Whole-token phrase matching prevents substring accidents. Because both methods receive identical intent, differences can be attributed to representation and judgment rather than a richer policy description.

3.3. Typed inventory and evidence

An `InventoryItem` contains normalized text and zero or more typed media records. Media records carry identifiers, dimensions, and local paths. A `Verdict` contains an action, severity, clause identifiers, evidence, confidence, tier, policy version, and display-only rationale. Text evidence is an exact quote with $[s, e)$ offsets. Image evidence is a bounding box (x, y, w, h) tied to one media identifier.

G1 verifies that each text slice reproduces its quote and each image box lies within the named frame. G2 resolves every clause identifier against the active policy version. G4 requires action to equal the fixed severity lookup. G6 refuses low-confidence ALLOW and detects disagreement across effort levels. G3 protects policy revision by requiring logged approval before a frozen holdout item can change from BLOCK to ALLOW. G5 validates explicit run budgets. The full gate definitions and coercions live in `src/adjacency/gates.py`.

The chain runs all applicable gates even after a failure. This creates a complete ledger and lets the strictest coercion win. The key property is non-interference from generated prose. No gate reads `rationale`. A convincing explanation cannot change an action, repair a nonexistent clause, or move a bounding box.

3.4. Confidence-gated escalation

The judge implements a fixed ladder rather than sending every item to the most expensive call. Tier 0 is a deterministic clean-text decider. Residual items receive a low-reasoning structured verdict at Tier 1. The system escalates to Tier 2 when confidence falls below the configured floor, severity lands on the review boundary, media accompanies text that did not trigger the baseline, a low-tier gate fails, the model call fails, or near-duplicate cluster members disagree. Tier 2 uses high reasoning and receives the low-tier verdict as auditable context, not as trusted truth.

For item i, let $r(i) \in \{0, 1, 2\}$ be its final tier and let c_j be the measured cost of model call j. The observed blended cost over corpus D is

$$\mathrm{Cost}_{1000}(D) = \frac{1000}{|D|} \sum_{j \in \mathrm{calls}(D)} c_j. \tag{2}$$

The implementation records provider-reported token usage, reasoning tokens, elapsed time, response identifier, and computed cost for every call. Tier 0 contributes no model call. The content-addressed fixtures permit exact replay, while the live report preserves original measurements.

3.5. Autopsy and the failure path

For each item, the baseline action and engine action are reduced to a delivery disposition. ALLOW serves. BLOCK and REVIEW withhold. Equal dispositions are AGREE. A baseline withhold paired with an engine allow is OVER_BLOCK. A baseline allow paired with an engine withhold is UNDER_BLOCK. This definition avoids treat-

ing BLOCK and REVIEW as if one served while the other withheld.

The Gradio Autopsy streams those three columns from committed artifacts. Selecting a row opens the source content, cited image region, final verdict, model-call measurements, and complete gate chain. One recorded item first appears as ALLOW, then fails G1 because its claimed text span is absent. The row turns red, the final action becomes REVIEW, and the item enters the human queue. The exact beat is in `artifacts/ui/autopsy_snapshot.json`.

The default queue is in-process. An environment switch can opt into Temporal Cloud for the review queue alone. That adapter defines one workflow, one human-adjudication signal, and one queue-state query in `src/adjacency/temporal_hitl.py`. The judge, policy compiler, corpus fetcher, gates, and demo do not move to Temporal. If the cloud service is unreachable, the default demo path does not import the SDK or connect.

4. Evaluation

4.1. Protocol and evidence boundary

The frozen corpus manifest is `corpus/frozen/manifest.json`. It binds each item, source URL, local media digest, and the overall corpus hash. The comparison baseline, compiled policy, judge traces, and raw-prompt runs carry the same policy and corpus hashes. Evaluation reads committed fixtures by default. Network calls are not needed to reproduce the gate and comparison logic.

The study asks four questions. First, do deterministic gates catch structural failures that should never be admitted? Second, does a free-text prompt produce grounded and repeatable decisions on the same inputs? Third, how does the implemented reasoning ladder route cost? Fourth, does a review survive a real durable queue transition? Table 1 reports only observed answers and places the authoritative repository path beside every value.

4.2. Seeded structural faults

The synthetic suite begins with known-good verdicts grounded in the compiled policy. It injects three fault families: a hallucinated text span, an invented clause identifier, and an incoherent severity-action pair. Each mutation declares its expected gate code. A case counts as caught only if that code appears, which prevents a coincidental failure elsewhere from inflating the result.

The run caught 12 of 12 injected cases. It rejected 0 of 4 unmodified controls. These figures come from `artifacts/tuesday/synthetic_faults.json`, which also stores the seed, per-case mutation, expected code, observed codes, and policy hash. This is the strongest evaluation result in the project because it is deterministic, reproducible, and independent of human labels. It is not a detection-accuracy claim. It tests gate behavior under known structural corruption.

4.3. Raw-prompt baseline

The baseline receives the same advertiser prose and the same ordered frozen corpus in two calls. It returns one action and one evidence claim per item, but without the compiled policy contract and post-call admission gates. A strict checker then asks whether each submitted span reproduces the quoted text at the claimed offsets.

Run 1 failed that grounding check on 68% of claims. The cross-run action disagreement rate was 2%. Both figures are in `evals/prompt_baseline/comparison.json`. The first result should not be misread as an average across runs. It is the observed rate for run 1. The comparison artifact also retains the other run's grounding result, every failed item, both response identifiers, token usage, cost, policy prose hash, and corpus hash.

The action disagreement is small in absolute terms, but it answers a different question than grounding. The prompt can often choose the same action while providing evidence that does not exist where it says it does. A system that checks only label agreement would miss the larger structural failure. This separation is the reason ADJACENCY treats evidence as a first-class contract.

4.4. Escalation economics

The observed ladder is bimodal rather than smooth. The frozen run placed 26 of 50 items at Tier 0, 2 of 50 at Tier 1, and 22 of 50 at Tier 2. Those counts and fractions are in `artifacts/wednesday/judge_report.json`. Tier 1 is a narrow residual in this corpus. A chart that visually implies even progression would be misleading.

Tier 2 therefore handled 44% of items. The measured blended cost was $6.12 per 1,000 decisions, rounded from the higher-precision value stored in `artifacts/wednesday/judge_report.json`. This is a routing and cost measurement for the frozen run. It is not a cost forecast for X traffic and not an accuracy-versus-cost curve. Accuracy requires independent labels that do not exist in this pre-event corpus.

4.5. Durable human review

The live integration test connected to Temporal Cloud, started the committed review workflow, queried a PENDING state, sent the adjudication signal, queried ADJUDICATED, and observed the workflow complete. The credential-free record is `artifacts/temporal/`

Table 1. Observed results. Every value is paired with its authoritative committed artifact. The raw-prompt grounding result is explicitly run-specific. The durable HITL row is protocol evidence, not a human label.

Question	Observed result	Authoritative artifact
Do gates reject seeded structural faults?	12/12 injected faults caught, with 0/4 clean cases rejected	`artifacts/tuesday/synthetic_faults.json`
Is the raw-prompt evidence grounded?	68% failure in run 1	`evals/prompt_baseline/comparison.json`
Do identical-input prompt runs agree on action?	2% action disagreement	`evals/prompt_baseline/comparison.json`
How much inventory reaches high reasoning?	44% at Tier 2	`artifacts/wednesday/judge_report.json`
What is the measured blended judge cost?	$6.12 per 1,000 decisions	`artifacts/wednesday/judge_report.json`
Did durable review complete?	PENDING query, adjudication signal, ADJUDICATED query, completed workflow	`artifacts/temporal/hitl_live_proof.json`

`hitl_live_proof.json`. It retains execution identifiers, timestamps, the state transition, a namespace hash, source artifact paths, and an empty list of recorded credential fields.

The supplied adjudication is labeled `integration-proof`. It is not a human label and must not enter a training dataset. The result establishes protocol behavior only. It does show that the failure path can leave a process, wait durably, and continue after a signal while keeping the default demo independent of the cloud backend.

5. Operational Design and Quality

Offline-first demonstration. Demo mode is the only UI mode. It loads the committed corpus, policy, baseline, traces, and economics assumption. It does not read an API key. Missing, hash-mismatched, or incomplete artifacts fail during application construction rather than halfway through a presentation. The over-block dollar counter is explicitly illustrative. Its assumption is in `artifacts/ui/economics_assumption.json` and is never presented as measured revenue.

Asymmetric testing. Contracts and gates are held to complete line and branch coverage in CI because they define admission. I/O surfaces use fixture-replay integration tests with one success path and one failure path per component. This is not uniform coverage by design. A model-layer bug still produces structured output that must cross the same gates. The policy and enforcement commands are documented in `QUALITY.md` and `.github/workflows/ci.yml`.

Credential boundary. External credentials are read from the environment only by explicitly selected live paths. Fixture recording rejects credential-shaped fields. The Temporal proof stores a namespace hash but not the raw namespace, address, or API key. The public demo uses no model or workflow credential.

Read-only rollout. ADJACENCY produces inspection and shadow evidence. It never mutates bids, budgets, or serving state. A credible deployment sequence starts with read-only autopsy, then shadow evaluation, then per-clause activation after independently labeled validation. G3 remains active during policy revision to stop silent loosening on a frozen holdout.

6. Limitations

The frozen corpus is an engineering fixture, not a representative sample of X inventory. It was selected to exercise multimodal and policy behavior, and its manifest contains the exact membership. No independent human labels exist before the event. This report therefore makes no precision, recall, false-positive, or detection-accuracy claim.

The keyword baseline is not X's internal blocklist. It is generated from the exact advertiser prose so that the comparison is controlled, but it cannot represent X's proprietary models, enforcement logic, or operational tuning. The delta columns describe disagreement between two implemented representations of the same intent. They do not estimate

production uplift.

The raw-prompt study has only two identical-input runs over one frozen corpus. It demonstrates an observed failure mode, not a universal rate for Grok 4.5. Hosted models may change. The fixtures make this run reproducible, but later live behavior must be measured again.

The cost result is tied to the recorded route distribution, model pricing, and call shapes. It does not include platform integration, storage, review labor, caching changes, or future pricing. The ladder also routed very little work to Tier 1, so additional tuning may be needed before describing it as a smooth cascade.

The Temporal exercise proves one protocol transition. It does not measure long-duration retention, load, retry storms, regional failure, or reviewer operations. Temporal remains opt-in. The in-process queue is the only backend the demo touches.

Finally, synthetic-media language is represented as a policy route to REVIEW. The project does not train or evaluate a synthetic-media detector and makes no detection claim.

7. Conclusion

ADJACENCY treats a model response as a proposal, not a decision. Advertiser prose becomes a versioned policy whose clauses point back to their source. Multimodal judgments become typed records whose evidence can be checked. Confidence controls where extra reasoning is spent. Deterministic gates decide what is admissible, and failures move toward review. The same prose produces the comparison blocklist, so the baseline is not intentionally weakened.

The evaluation supports the design claim at the level this artifact can honestly sustain. The seeded fault suite catches its targeted structural corruptions. The tested raw prompt is poorly grounded in one run and not fully stable across identical inputs. The ladder exposes its actual high-reasoning fraction and blended cost. The durable queue completes a real cloud transition. Each statement points to a committed artifact rather than a remembered number.

The next scientific step is independent labeling, not a larger model claim. A held-out, doubly labeled corpus can measure risk and coverage for each tier, quantify the cost of review, and test policy revisions under G3. Until then, the system remains what it should be: a read-only autopsy that makes model failure visible before anyone asks it to control delivery.

Impact Statement

Brand-safety systems can affect expression, monetization, advertiser trust, and reviewer workload. An over-broad filter can suppress safe inventory. An under-broad filter can place advertising beside content a brand intended to avoid. ADJACENCY makes both directions visible and abstains when its evidence or structure fails. That design reduces silent automation, but it does not remove judgment from policy writing or human adjudication. Any deployment should measure subgroup and language effects, publish reviewer guidance, provide appeal paths, and keep policy changes auditable. The current system has no write access to ads and should not receive it without independent evaluation.

References

Chen, L., Zaharia, M., and Zou, J. How is ChatGPT's behavior changing over time? *arXiv preprint arXiv:2307.09009*, 2023a. URL https://arxiv.org/abs/2307.09009.

Chen, L., Zaharia, M., and Zou, J. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023b. URL https://arxiv.org/abs/2305.05176.

Geifman, Y. and El-Yaniv, R. SelectiveNet: A deep neural network with an integrated reject option. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pp. 2151–2159, 2019. URL https://proceedings.mlr.press/v97/geifman19a.html.

Kiela, D., Firooz, H., Mohan, A., Goswami, V., Singh, A., Ringshia, P., and Testuggine, D. The hateful memes challenge: Detecting hate speech in multimodal memes. In *Advances in Neural Information Processing Systems*, volume 33, 2020.

Lykouris, T. and Weng, W. Learning to defer in content moderation: The human-ai interplay. *arXiv preprint arXiv:2402.12237*, 2024. URL https://arxiv.org/abs/2402.12237.

Mozannar, H. and Sontag, D. Consistent estimators for learning to defer to an expert. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pp. 7076–7087, 2020. URL https://proceedings.mlr.press/v119/mozannar20b.html.

Ribeiro, M. T., Wu, T., Guestrin, C., and Singh, S. Beyond accuracy: Behavioral testing of NLP models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 4902–4912. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.acl-main.442. URL https://aclanthology.org/2020.acl-main.442/.

Twitter, Inc. Annual report on form 10-k for the fiscal year ended december 31, 2021. U.S. Securities and Exchange Commission, 2022. URL https://www.sec.gov/Archives/edgar/data/1418091/000141809122000029/twtr-20211231.htm. Accessed 2026-08-03.

X Corp. Brand safety at x. X Business Help Center, 2026. URL https://business.x.com/en/help/ads-policies/brand-safety. Accessed 2026-08-03.

xAI. Grok 4.5. xAI Developer Documentation, 2026a. URL https://docs.x.ai/developers/grok-4-5. Accessed 2026-08-03.

xAI. Reasoning. xAI Developer Documentation, 2026b. URL https://docs.x.ai/developers/model-capabilities/text/reasoning. Accessed 2026-08-03.